

A Review of Screenshot-to-Code Generation Systems

Padamjit Sandhu
Software Engineer, PayPal, USA
sandhp7@gmail.com

Pankaj Khurana
VP Technology & Consulting, Rocket, USA
punnykhurana@gmail.com

Jimmy Singla
Lovely Professional University, INDIA
dr.jimmy.cse@gmail.com

Abstract: The conversion of screenshots into source code is very important phase in the automation of user interface development. There are artificial-based tools which can understand images and further convert them into executable source code. These system's performance is further improved with the incorporation of generative artificial intelligence. These generative artificial intelligence-based systems achieve high accuracy in converting sketches into source code. Further these systems are helpful for developers, otherwise it was time consuming and costly to convert images into source code and developers often distracted from focusing on logic and functionality. This review paper presents the existing studies which generate source code from sketches using artificial intelligence and machine learning. In this paper, several studies are evaluated and compared based on different parameters. Their strengths and weaknesses are also discussed in this paper. This review paper will help the researchers and authors working in this area to further improve GUI design process and in development of more accurate systems.

Keywords: Graphical User Interface, Machine learning, Code Generation, Review study, conceptual images, GUI design sketches.

I. INTRODUCTION

The software engineering process is potentially enhanced by AI, which is quickly transforming the procedure for creating software. Owing to equipment, techniques, and developments that increase effectiveness, flexibility, and responsiveness, the application development sector is evolving quickly [1]. Screenshot to code generation represents one of the most important and revolutionary developments made possible by AI. Effective graphical user interfaces (GUIs) are essential to contemporary software platforms because it make jobs easier and more efficient to complete while also keeping users fascinated [2]. The development of software is a challenging and intricate procedure that includes phases for evaluation and coding. But additionally, it has been shown that it is a costly process, particularly in situations that follow protocols, guidelines, and team frameworks [3]. According to a research study of over five thousand software developers, 51% of the respondents said they worked on application user interface design duties every day, which is more frequently

than other development jobs. Moreover, the user interface consumes 50% of the development time, while it accounts for 48% of the code for the software on average [4].

In software development, screenshot to code creation becomes more and more crucial, particularly with the application of artificial intelligence (AI) and machine learning (ML) [5]. Iterative prototyping is used in the design of GUIs, beginning with the creative method of creating drawings, which are basic handmade representations. It is a useful pictorial method for quickly, easily, and affordably communicating ideas, discussing them, and comparing various options. In the screenshot to code generation process, Sketches are used to develop wireframes, which specify the order of appearance and depict the framework and arrangement of the user interface, excluding any graphical elements like colors or pictures [6-8]. The design of the graphics improves the wireframe when it has been constructed and modified, eventually turning it into a highly detailed prototype. Once the design is complete, it is executed and generates the end result as shown in Figure 1 [9]. When designing a user interface, these phases might be performed repeatedly as part of an iterative prototype process, necessitating redesign anytime modifications are implemented. Hence, the procedure of creating code from GUI designs has an enormous opportunity for automation because it is a laborious and inefficient procedure [10-12]. In the last few years, machine learning techniques have been used to automatically generate GUI code. It concentrates on creating front-end part code utilizing a transformer-powered machine learning algorithm [13]. The goal of Screenshot-to-Code generation approaches is to automate tedious and frequently performed processes so that developers can concentrate on more complex development and solving issues [14-15]. Due to notable developments in this field, scholars have recently investigated the application of AI, especially DL techniques, for the Screenshot-to-Code generation task. Hence, this review paper presented a systematic evaluation which mainly focuses on the various machine learning systems which are developed by different authors or scholars for automating the GUI code from images.

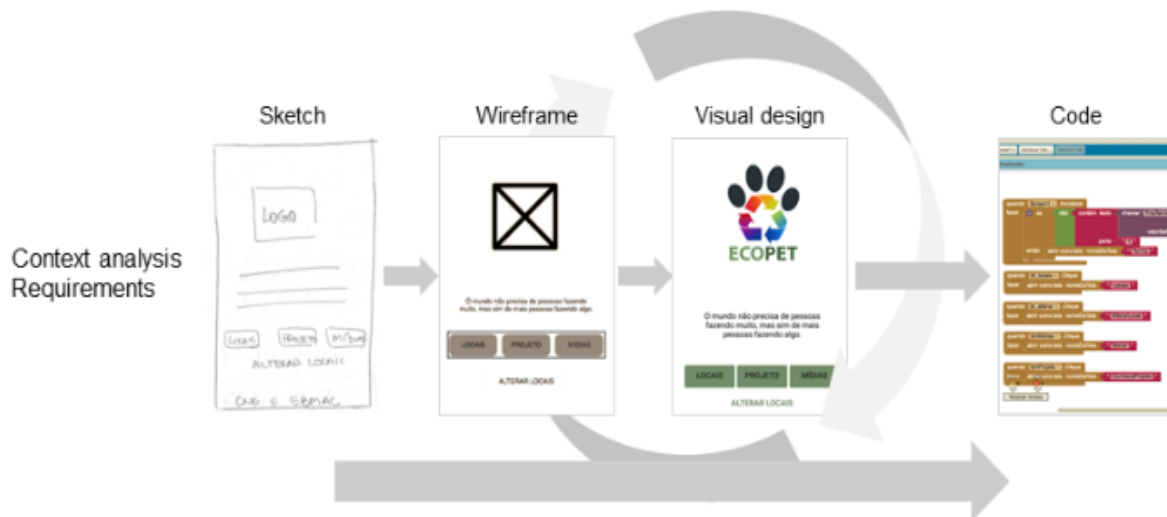


Fig. 1. Flow for screenshot to code generation process [9]

The key conclusions from the examination that may be useful to both academics and professionals have been gathered in this paper.

II. RELATED WORK

In this section, various technologies and methodologies are summarized that assisted in the development of screenshot to code generation. This investigation includes the different approaches of artificial intelligence utilized to generate the GUI from wireframes, sketches or screenshots, the main motivation of the author, what are the results of the proposed methodology by authors and how it impacts in the advancement of screenshot to code generation.

Cai et al. (2023) presented an automatic graphical user interface code generator (GUICG) for Android that effectively identifies components of UI pictures and produces the required source code for front end by combining image processing and deep neural networks methodologies. Using the combination of deep learning-based image processing and target detection algorithms, the initial phase of the developed platform is identifying and categorizing GUI elements into different kinds, such as pictures and buttons and expressing these elements as particular GUI objects. Furthermore, a code generator is used in its 2nd phase, which produces the matching code. A Rico dataset is used to collect the screenshots, which are used as input and generate a human readable code. This development attained 57.93 percent F1 in text detection and 54.3 percent in the detection of elements.

Pranav et al. (2023) focused on how well a website's frontend can be built so that developers working on the backend are able to evaluate their relational database processes without having to spend a lot of time on frontend development. In the developed system, LLMs are used in

generative AI to automatically produce Java source code. The main goal of this suggested system is to identify every component in sketch pictures and input the attributes that are retrieved to the storm breaker algorithm that helps in producing an HTML webpage in accordance with the specifications. With cutting-edge technology, this system operates better than other systems. After the model was trained on 145 sketch pictures across 100 epochs, the mean average precision was 96%.

Wan et al. (2024) developed DCGen, a divide and conquer strategy, in order to automate the conversion of the layout of websites to user interface code. In this model, screenshots are first broken up into comprehensible sections by DCGen, which then creates explanations for every section and reassembles them to a full user interface associated with the provided screenshot. A data set of thousands of extremely popular genuine web pages was assembled for the investigation. Using a sample of 120 websites from the gathered dataset, the authors assess the efficacy of the used methodology across multiple state of the art multimodal large language models (MLLMs). It shows that the developed model outperforms rival techniques in terms of visual similarity by up to 14 percent. As an output, this model generates user interface code in HTML and CSS files corresponding to the provided structure and layout of screenshot.

Xiao et al. (2024) designed a comprehensive system, Prototype2Code, which uses user interface prototypes at automatically create code. In order to improve the design quality and strengthen the durability of generated code, it incorporates prototype cleaning as an essential stage in the whole procedure. This study offers an entire workflow that reduces developers' repetitious work by turning precise GUI mock-ups into front end code that is ready to deployment. The authors used a graph neural network (GNN) to identify

and categorise design layers and a large language model (LLM) in order to generate the required source code. The results of this study show that the developed platform has a lower error rate as compared to other existing frameworks.

Vargas-Monroy et al. (2025) presented a framework which helps in the designing phase of a project and solved the suggested issues by using deep learning and Model-Driven Architecture (MDA) to create source code using architecture drawings and the corresponding test scenarios. By utilizing MDA, the proposed platform successfully connects the stages of development and execution and offers a smooth way to match practical demands with company rationale. According to the results of this study, with error rates of no more than five percent, the working model processed 150 drawings in an hour on average and attained an accuracy of 85 percent in image interpretation. The system provides domain specific language (DSL) as an output, which is then converted into source code using plugins. Moreover, a GitHub repository of created code is generated by the system and also allows the created code to be deployed to Heroku. Hence, as a result, the proposed platform provides complete code artifacts as an end result.

Li et al., (2025) introduced a novel system named as Sketch2Code for utilizing the abilities of Vision Language Model (VLM) to create a web user interface from the drawings provided by the customer. An interactive agent framework is implemented in this system, which included two modes such as asking questions and getting feedback. A Large Language Model (LLM) is used to simulate this interaction with the user. After interpreting the sketches, the model generates a source code in HTML and CSS, which is a kind of prototype. This prototype source code is refined by the proposed system by getting feedback and asking questions to the user, and produces an updated HTML and CSS source code file. Moreover, the performance is evaluated by measuring layout interaction over union (IoU) and visual similarity, which enhanced up to +2.7 percent and +7.1%, respectively.

Si et al. (2025) introduced a web platform named Design2Code using multimodal large language models. In order to evaluate the extent to which existing MLLMs can produce code implementation which immediately transforms the provided input (screenshots) into the required and sample web pages, a set of 484 different authentic websites is selected as instances for testing and create a collection of predetermined criteria. Hence, an automatic evaluation metric is introduced which examines the matching of low-level elements like colour, position, text and visual similarity. As a result, it is observed that some frameworks frequently have difficulty retaining visual aspects, such as these models fail to notice or include certain GUI components. Because it integrates both automated and human-based assessment for

practical, diversified websites, the developed tool is helpful for the academic area to gauge development.

Chen et al. (2025) proposed a brand new vision-based auto-correcting and hierarchical method, DesignCoder, which uses design mock-ups to produce superior GUI coding. Using a multimodal cognitive thinking platform, the developed system improves MLLM's comprehension of user interfaces by successfully identifying complicated layered designs. The proposed solution is specifically designed for mobile user interface as it supports React Native. In this study, the metadata and pictures of design are gathered to make a component tree, which further obtained the outcome in the form of code supporting React Native. The result of this study demonstrates that the proposed approach improved the visual similarity matrices with 12.82, 9.52, and 37.63 percent in SSIM, CLIP and MSE, respectively.

Gui et al. (2025) developed a framework known as LaTCoder, which assists in generating codes from designs and develops a front-end web page using MLLMs. The Chain of Thoughts approach of human thinking is used as a motivation to introduce layout as thought (LaT). The mock-up designs are provided to the system as an input, which are further categorized into different segments. The applied methodology of MLLM is then used in code generation in HTML and CSS. The result of this research study shows that the overall performance of the developed system is enhanced by 66.67 percent and reduced mean absolute error by 38%.

Gui et al. (2025) examined that the majority of current platforms designed using MLLM are limited and also do not use reliable and vast data sets from the real world, which further leads to poor performance in generating code from designs automatically. Therefore, this research work presented WebCode2M, an entirely novel data set of 2.56M data samples, every instance of this dataset includes design images, code of the web page and information related to page structure. This dataset is used to create a source code in HTML and CSS from screenshot images using MLLM methodology, WebCoder. The experimental results of this study show that it substantially increases MLLM's capacity to produce code from screenshots

Table 1 provides a comparison between different platform or frameworks developed by several authors from 2020 to 2025. This comparison has been done including the reference of the selected research article along with author name and year of publication, the platform type (Android, web or IoT based framework), artificial intelligence methodologies used to develop a source code, input provided to developed systems, output generated by the models and their end results which indicates model's accuracy or improvement as compare to other models. In this comparison, only those research papers are selected that use either sketches or images or drawings or mock-up designs as an input to generate source code in the required format, such as HTML/CSS or any other native code

which is understandable. The code generation models which use text alone as an input are excluded from this systematic review study. The applications considered in existing models are software testing automation, mobile and web app development. The major strength of existing models is high level of automation in converting screenshots into source code and the limitations of existing models are that they

depend on large datasets and the functional validation of generated code is also missing in few models. LLM based models also have very high computational cost. The scope of work includes the deployment of front -end-source code with the help of artificial intelligence from UI-sketches, mock-ups, wireframes and screen shots.

TABLE I. COMPARISON OF VARIOUS PLATFORMS INCLUDING THEIR METHODOLOGY, INPUT, OUTPUT AND RESULTS

Reference	Platform	Methodology	Input	Output	Results
Cai et al. (2023) [16]	Android	Image processing and deep neural networks	GUI images from the Rico dataset	Android XML code	57.93% F1 in text detection and 54.3% in the detection of elements
Pranav et al. (2023) [17]	Web	LLM	Sketch pictures	Java source code	The achieved mean average precision is 96%.
Wan et al. (2024) [18]	Web	MLLM	Screenshots	HTML and CSS file of code	Improve visual similarity by up to 14 percent
Xiao et al. (2024) [19]	Web	GNN and LLM.	Design prototype	HTML and CSS code	Low error rate as compared to existing approaches
Vargas-Monroy et al. (2025) [2]	Web	Deep Learning and MDA	Diagram images with text	Complete code artefacts	Achieved an Accuracy of 85% in image interpretation
Li et al., (2025) [20]	Web	LLM and VLM	Sketches	HTML and CSS source code	Improved visual similarity by 7.1% and layout IoU by 2.7%
Si et al., (2025) [21]	Web	MLLM	Screenshots	Front end source code in HTML and CSS	Compared different models and GPT-4V outperformed
Chen et al. (2025) [22]	Mobile platform based	MLLM	Design pictures	React Native Code	Improved visual similarity matrices as compared to other systems.
Gui et al. (2025) [23]	Web	MLLM	Mock-up designs	HTML/CSS code	Improved performance by 66.7% and reduced Mean Absolute Error by 38%
Tankala et al. (2023) [24]	Web	CNN and RNN	Mock-ups	HTML code	Achieved 96% accuracy in 200 epochs
Gui et al. (2025) [25]	Web	MLLM	Screenshot images	Code in HTML and CSS	Substantially increases MLLM's capacity to produce code from screenshots
Wu et al. (2024) [26]	IoS	LLM	User interface explanation in natural language	SwiftUI code	Improves performance from 0.03 to 0.79 approximately

III. CONCLUSION

Front-end code generation from picture blueprints, including screenshots and sketches, is a difficult operation which necessitates an accurate comprehension of the pictures in

order to identify user interface components along with its structure. In order to automatically produce source code and speed up the GUI designing phase, this literature review offers a review of numerous methodologies and tools which employ diverse technologies of artificial intelligence, such as deep learning and machine learning. As per the examination,

it is clearly observed that the deep learning techniques demonstrated to be more accurate as well as appropriate to generate source code from any kind of picture. These methods showed notable improvements in code production, effectiveness and quality of code. Even if artificial intelligence methods have a lot of promise for automated generation of code, but it has drawbacks and difficulties too. Among the major issues that must be resolved are the data accessibility, which is used to train any intelligent model and after that, reliability, effectiveness and ability to understand the code generated by AI-based tools. Therefore, in future, instead of enhancing the performances of models, this technology must establish strong security measures, guarantee transparency and accountability and encourage human and AI cooperation in development procedures.

REFERENCES

- [1] Mathavan, A. S. (2025). Automated Code Generation Using Artificial Intelligence: Opportunities, Challenges, and Ethical Considerations for the Future of Software Development. *Journal of Science Technology and Research (JSTAR)*, 6(1), 1–12.
- [2] Vargas-Monroy, G., Gonzalez-Roldan, D.-B., Montenegro-Marín, C. E., Daza-Corredor, A.-P., & Leal-Lara, D.-D. (2025). Code generation system based on MDA and Convolutional Neural Networks. *Frontiers in Artificial Intelligence*, 8. <https://doi.org/10.3389/frai.2025.1491958>
- [3] Odeh, A., Odeh, N., & Mohammed, A. S. (2024). A comparative review of AI techniques for Automated Code Generation in software development: Advancements, challenges, and future directions. *TEM Journal*, 13(1), 726–739. <https://doi.org/10.18421/tem131-76>
- [4] Zaki Gad, M. S., Marie, M., & Sayed Yakoub, A. E. (2024). A Survey of techniques for Automatic Code Generation from User Interface Designs with Various Fidelities. *Informatics Bulletin, Faculty of Computers and Artificial Intelligence*, 6(2), 69–81.
- [5] Alenezi, M., & Akour, M. (2025). AI-Driven Innovations in Software Engineering: A review of current practices and Future Directions. *Applied Sciences*, 15(3), 1344. <https://doi.org/10.3390/app15031344>
- [6] Gudoniene, D., Staneviciene, E., Buksnaitis, V., & Daley, N. (2023). The scenarios of Artificial Intelligence and wireframes implementation in engineering education. *Sustainability*, 15(8), 6850. <https://doi.org/10.3390/su15086850>
- [7] Riaz, S., Arshad, A., S. Band, S., & Mosavi, A. (2022). Transforming hand drawn wireframes into front-end code with Deep Learning. *Computers, Materials & Continua*, 72(3), 4303–4321. <https://doi.org/10.32604/cmc.2022.024819>
- [8] Calò, T., & De Russis, L. (2025). Advancing code generation from visual designs through Transformer-based architectures and specialized datasets. *Proceedings of the ACM on Human-Computer Interaction*, 9(4), 1–37. <https://doi.org/10.1145/3734190>
- [9] de Souza Baule, D., von Wangenheim, C. G., von Wangenheim, A., & Hauck, J. C. R. (2020). Recent progress in automated code generation from GUI images using machine learning techniques. *Journal of Universal Computer Science*, 26(9), 1095–1127.
- [10] Das, A., Chowdhury, S., Ghosh, D., Saha, S., Chakraborty, S., Das, D., Bhattacharyya, S. K., Bhattacharyya, S., Choudhury, P. D., & Dutta, K. N. (2025). Automated code translation: Enhancing efficiency and accuracy across programming languages. *Advances in Intelligent Systems Research*, 736–748. https://doi.org/10.2991/978-94-6463-787-8_55
- [11] Vemuri, V. P. K. (2024). The Evolving Landscape of Front-End Engineering in the age of Generative AI and Autonomous Systems. *Journal of Artificial Intelligence, Machine Learning and Data Science*, 2(1), 1–7.
- [12] Rosyida, H. F., Syafei, A., & Nuha, M. U. (2025). The evolving field of graphic design: Challenges and opportunities in the integration of Artificial Intelligence. *International Journal of Graphic Design*, 3(1), 19–38. <https://doi.org/10.51903/ijgd.v3i1.2537>
- [13] Cai, B., Luo, J., & Feng, Z. (2023). A novel code generator for graphical user interfaces. *Scientific Reports*, 13(1). <https://doi.org/10.1038/s41598-023-46500-6>
- [14] Wan, Y., Wang, C., Dong, Y., Wang, W., Li, S., Huo, Y., & Lyu, M. (2025). Divide-and-conquer: Generating UI code from screenshots. *Proceedings of the ACM on Software Engineering*, 2(FSE), 2099–2122. <https://doi.org/10.1145/3729364>
- [15] Calò, T., & De Russis, L. (2025a). Advancing code generation from visual designs through Transformer-based architectures and specialized datasets. *Proceedings of the ACM on Human-Computer Interaction*, 9(4), 1–37. <https://doi.org/10.1145/3734190>
- [16] Cai, B., Luo, J., & Feng, Z. (2023a). A novel code generator for graphical user interfaces. *Scientific Reports*, 13(1). <https://doi.org/10.1038/s41598-023-46500-6>
- [17] Pranav, T., Prathamesh, P., Koushik, P., & Sankalp, W. (2023). Automated HTML Code Generation on Sketch Images Using Storm Breaker Algorithm. *Technix International Journal for Engineering Research*, 10(12).
- [18] Wan, Y., Wang, C., Dong, Y., Wang, W., Li, S., Huo, Y., & Lyu, M. R. (2024). Automatically Generating UI Code from Screenshot: A Divide-and-Conquer-Based Approach. *arXiv Preprint arXiv:2406.16386*.
- [19] Xiao, S., Chen, Y., Li, J., & Chen, L. (2024). Prototype2Code: End-to-end Front-end Code Generation from UI Design Prototypes. *Proceedings of the ASME 2024 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*.
- [20] Li, R., Zhang, Y., & Yang, D. (2025). Sketch2Code: Evaluating vision-language models for interactive web design prototyping. *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 1, 3921–3955. <https://doi.org/10.18653/v1/2025.naacl-long.198>
- [21] Si, C., Zhang, Y., Li, R., Yang, Z., Liu, R., & Yang, D. (2025). Design2Code: Benchmarking Multimodal Code Generation for Automated Front-End Engineering. *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, 1, 3956–3974.
- [22] Y. Chen et al., “DesignCoder: Hierarchy-Aware and Self-Correcting UI Code Generation with Large Language Models,” *arXiv preprint arXiv:2506.13663*, 2025.
- [23] Y. Gui et al., “LatCoder: Converting webpage design to code with layout-as-thought,” *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, vol. 2, pp. 721–732, Aug. 2025. doi:10.1145/3711896.3737016
- [24] D. K. Tankala, K. Bhuvaneshwari, M. Anusha, P. V. Rishitha, and S. U. Nazi, “Automatic HTML Code Generation from Mock-up Images,” *Journal of Engineering, Computing and Architecture*, vol. 13, no. 9, pp. 224–232, 2023.
- [25] Y. Gui et al., “WebCode2M: A Real-World Dataset for Code Generation from Webpage Designs,” *Proceedings of the ACM on Web Conference 2025*, pp. 1834–1845, 2025. doi:https://doi.org/10.1145/3696410.3714889
- [26] J. Wu et al., “UICoder: Finetuning Large Language Models to Generate User Interface Code through Automated Feedback,” *arXiv preprint arXiv:2406.07739*, 2024.